SPRINT 01, FUNDAMENTOS DO PROCESSO

Gestão de configuração (SCM)

Versionar código, dependências, configuração e ambiente: o chão comum que faz o resto do DevOps funcionar.

Aula	02 de 14
Data	27 de abril de 2026, segunda-feira, 10h00 às 12h00
Professor	José Romualdo da Costa Filho
Convidado	Arquiteto WSO2, hands-on de OpenChoreo
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

Bloco	Atividade	Duração
10h00, Daily Time	Abertura com cronômetro de 15 min, status rápido por grupo	15 min
10h15, Ponderada	Briefing da atividade ponderada da sprint e alinhamento da rubrica	5 min
10h20, Ativação	Recuperação dos auto-estudos, três quizzes rápidos em plenário	15 min
10h35, SCM na prática	O que SCM versiona, estratégias de branching, commits e configuração	30 min
11h05, C4 e reprodutibilidade	C4 Model como linguagem comum, Docker e Kubernetes essenciais	20 min
11h25, OpenChoreo ao vivo	Sessão com o arquiteto da WSO2, conceitos e demo coletiva	25 min
11h50, Síntese	Quizzes de fechamento, homework e ponte para Aula 03	10 min

DAILY, 15 MIN

Daily Time

Alinhamento rápido de cada grupo: o que avançou desde a última aula, o que trava e qual é a próxima meta.

15:00

INICIAR PAUSAR ZERAR

PONDERADA, SPRINT 02, INDIVIDUAL, 4 HORAS

JactoDesk: suba três Components no OpenChoreo

Um mesmo minimundo para toda a turma. Você parte de um repositório-semente que roda, mas esconde vários problemas ligados ao que vimos hoje. Sua missão é achá-los, arrumá-los e implantar três Components (Service, Web Application e Scheduled Task) no OpenChoreo do arquiteto da WSO2.

REPOSITÓRIO-SEMENTE

`github.com/josercf/inteli-2026-1B-T13-ES10-Ponderada-S2`

Clone, crie um projeto no seu GitLab e dê push lá. É nele que seus commits, pipeline e correções vão viver.

COMO COMEÇAR

Leia o README, rode local, observe os comentários no código, acompanhe o pipeline do GitLab. O que falhar é pista. Descobrir o que precisa ser arrumado faz parte do exercício.

O QUE ENTREGAR

- Link do seu repositório no GitLab com as correções em commits bem feitos e pipeline verde.
- Vídeo de até 5 minutos descrevendo o que você corrigiu e por quê.
- Demo do OpenChoreo com os três Components rodando em dev.

O QUE CONTAR NO VÍDEO

Escolha os dois ou três problemas mais interessantes que você encontrou, mostre o commit que arrumou cada um e termine com a tela do OpenChoreo. Sem roteiro engessado, mas sem demo sem

Objetivos de aprendizagem

inteli.

Compreender

Reconhecer que SCM cuida de código, dependências, configuração e infraestrutura, não só de Git.

Comparar

Distinguir GitFlow, Trunk-Based e GitHub Flow e saber quando cada um cabe.

Modelar

Usar o C4 Model para descrever arquitetura em níveis de zoom apropriados.

Aplicar

Implantar uma aplicação no OpenChoreo e explicar, na defesa, cada decisão que vocês tomaram.

BLOCO 1, 15 MIN

Do que vocês leram

Ativação dos auto-estudos em três quizzes rápidos. Cada um vale uma discussão curta em plenário.

Ativação, 1 de 3, SCM

inteli.

Dos itens abaixo, qual não é tradicionalmente objeto da Gestão de Configuração de Software?

- O código-fonte versionado com histórico de quem alterou cada linha.
- As dependências externas com versões travadas em lockfile.
- O salário dos desenvolvedores que trabalham no repositório.
- Os arquivos de configuração de cada ambiente onde a aplicação roda.

Ativação, 2 de 3, Configuração

inteli.

Por que o artigo da Red Hat sobre gestão de configuração recomenda manter os parâmetros de ambiente fora do artefato final da aplicação?

- Porque a aplicação só aceita configuração em tempo de compilação.
- Para que a mesma imagem rode em dev, homologação e produção sem ser recompilada.
- Porque variáveis de ambiente consomem mais memória que arquivos.
- Porque a LGPD obriga configuração dentro do container.

Ativação, 3 de 3, C4 Model

inteli.

No C4 Model de Simon Brown, qual é o nível que mostra quais serviços da aplicação se comunicam entre si, sem descer no código?

- Contexto (Nível 1)
- Container (Nível 2)
- Componente (Nível 3)
- Código (Nível 4)

BLOCO 2, 30 MIN

SCM para além do Git

Git é a ponta do iceberg. Versionar de verdade envolve código, dependências, configuração e ambiente inteiro.

O que SCM versiona de verdade

inteli.

Muita gente reduz SCM a Git. Na prática, são quatro camadas que precisam ser versionadas juntas. Deixar qualquer uma de fora cria aquele deploy que "rodava na minha máquina" e some em produção.

01
Código
O que o desenvolvedor escreve. Arquivos-fonte, testes, scripts de build.

`Git, branches, tags`

02
Dependências
Bibliotecas externas com versões travadas. Sem lockfile, build de ontem não reproduz hoje.

`package-lock.json, poetry.lock`

03
Configuração
O que muda entre ambientes. Variáveis, feature flags, endpoints. Fora do código, mas versionada.

`ConfigMap, Secret, .env.example`

04
Ambiente
O SO, o runtime, a infra. Virou código: Dockerfile, manifesto Kubernetes, Terraform.

`Dockerfile, IaC, Helm`

A regra prática
Se algo que só existe no seu laptop impede um colega de rodar a aplicação, esse algo deveria estar versionado. Segredos são a única exceção: nunca vão para o repositório, mas o nome do segredo e o formato esperado vão.

Estratégias de branching, três modelos clássicos

inteli.

Como o time usa branches é uma das decisões mais visíveis do SCM. Três modelos dominam hoje. Cada um pressupõe um ritmo de entrega diferente.

2010, Vincent Driessen

GitFlow

- **main** guarda versões liberadas.
- **develop** integra novo código.
- Branches auxiliares: feature, release, hotfix.
- Release passa por merge formal antes de virar tag.

Bom para: produtos com versões numeradas, janelas de release, suporte a múltiplas versões em paralelo.

2011, Scott Chacon

GitHub Flow

- **main** sempre publicável.
- Feature branches curtas, merge via PR.
- Deploy pode acontecer em qualquer merge.
- Não existe develop, hotfix é mais uma PR.

Bom para: produtos web que entregam continuamente, times pequenos, SaaS.

Anos 2000, Google, Meta

Trunk-Based

- Todos commitam direto em **main** (trunk).
- Branches vivem horas, não dias.
- Feature flags controlam o que está exposto.
- Pipeline de CI com cobertura alta é pré-requisito.

Bom para: monorepos, equipes grandes, deploy contínuo com alta automação.

A escolha costuma refletir o ritmo do negócio. GitFlow protege contra release acidental, Trunk-Based remove atrito, GitHub Flow fica no meio.

SALA DE AULA INVERTIDA, 3 MIN EM DUPLAS

Qual fluxo para qual projeto?

Para cada cenário, decidam em dupla: **GitFlow, GitHub Flow ou Trunk-Based**, e escrevam a razão em uma frase.

1, Sistema embarcado de caixa eletrônico

Versões 4.2, 4.3 e 5.0 convivem em bancos diferentes. Cada banco atualiza no seu ritmo, todas precisam de patch de segurança.

`O que pesa: múltiplas versões vivas em paralelo.`

2, SaaS de gestão para dentistas

Time de 6 desenvolvedores, deploy 2 vezes por semana, clientes usam sempre a versão mais recente, sem opção de ficar atrás.

`O que pesa: uma versão só, entrega frequente.`

3, Monorepo com 80 devs

Vários times, build caro, dependências cruzadas. Branches longos viram pesadelo de merge, a automação cobre testes e segurança.

`O que pesa: escala do time e atrito de merge.`

4, App mobile com review na loja

Cada versão esperando aprovação da Apple. Paralelo a isso, o time continua codando a próxima, e às vezes precisa corrigir a versão em revisão.

`O que pesa: release bloqueado por terceiro.`

Commits e tags como contrato

inteli.

Branch é onde o código cresce. Commit é o registro de uma mudança. Tag é o ponto que diz "esta versão exata foi publicada". Juntos, eles formam o contrato mínimo de rastreabilidade.

Conventional Commits

Prefixo que classifica o tipo da mudança. Deixa o changelog legível e permite automação.

```
feat(auth): adicionar login por biometria
fix(pagamentos): tratar timeout do gateway
chore(deps): atualizar react para 19.1
# Breaking: BREAKING CHANGE no corpo
```

Proteção de main e tags

- **Branch protection:** PR obrigatório, pelo menos 1 revisor, status checks verdes.
- **Signed commits:** commits assinados com chave GPG ou SSH comprovam autoria.
- **Semantic versioning:** tag MAJOR.MINOR.PATCH. Breaking muda MAJOR.
- **CODEOWNERS:** define quem revisa cada pasta automaticamente.

O teste do incidente

Às 3h da manhã produção quebra. Você consegue, em menos de 5 minutos, identificar a tag que foi publicada, encontrar o commit responsável, abrir a PR que o mergeou e saber quem aprovou? Se sim, seu SCM está saudável.

Conventional Commits, o padrão que a ponderada cobra

inteli.

A spec está em `conventionalcommits.org/v1.0.0`. Cada commit vira um dado legível por máquina, que alimenta changelog, semantic release e auditoria.

Formato

```
<tipo>(<escopo>)!: descrição

[corpo opcional]

[rodapé opcional, BREAKING CHANGE]
```

Tipos aceitos no repo da ponderada: `feat, fix, docs, style, refactor, test, chore, ci, build, perf, revert`.

Como o repo cobra

- **Local:** Husky instala um hook `commit-msg` que chama o commitlint. Mensagem fora do padrão, commit bloqueado.
- **CI:** o job `lint:commits` roda em cada MR. Se um commit antigo passou pelo Husky mas não está no padrão, a MR falha e não merge.
- **Revisão:** o avaliador dá `git log --oneline` e sabe em trinta segundos o que você mexeu.

Boas mensagens, de forma geral

Prefixo claro (`fix` , `feat` , `chore`), escopo identificando qual parte do sistema foi tocada, descrição curta no imperativo, corpo opcional com o porquê. Um commit por mudança lógica, não um commitão com tudo.

Configuração: dentro ou fora do código?

inteli.

O **12-factor app** (Heroku, 2011) virou referência. O fator 3 é claro: config fica no ambiente, nunca no código. Mas nem toda configuração é igual.

Versionar no repo

Defaults seguros (.env.example), schemas de configuração, feature flags estáticas, ConfigMap de Kubernetes de dev.

Injetar no ambiente

Endpoints por ambiente, credenciais de serviços, flags dinâmicas, tamanhos de pool. Mesma imagem para todos, valores diferentes.

Nunca no repo

Senhas, tokens de API, chaves privadas, certificados. Vão para cofres (Vault, Secrets Manager, Sealed Secrets) e a aplicação os lê pelo ambiente.

Por que isso existe

Sem essa separação, promover uma imagem de dev para produção exigiria rebuild. Com ela, a mesma imagem aprovada em staging é a que vai para produção, e só a configuração muda. Isso é o que permite que a Aula 03 fale de métricas como lead time e deployment frequency com alguma base.

Segredos comitados: detectar e reagir

inteli.

O erro vai acontecer: alguém cola uma chave no código para destravar o dev na sexta, esquece de tirar no domingo, e comita na segunda. O que separa o time que aprendeu do time que repete o mesmo erro é o que vem depois. **Apagar o commit não resolve**, a chave já foi pública.

Detectar, TruffleHog

Ferramenta open source que varre o histórico do git procurando credenciais por padrão e por entropia. Roda como container, não precisa de instalação nativa.

```
docker run --rm -v "$PWD":/src \
  trufflesecurity/trufflehog:latest \
  git file:///src --fail
```

Reagir, rotacionar

- Rotacione a credencial no provedor. A chave comprometida morre.
- Só depois, decida se vale reescrever o histórico. Às vezes não compensa.
- Mova a leitura para variável de ambiente, entregue como Secret no OpenChoreo.
- Adicione o padrão ao pre-commit local e ao CI, para não repetir.

Se o pipeline da sua MR derrubar por TruffleHog

Não basta apagar a linha no próximo commit. Rotacione primeiro (no provedor, ou simulado se for um laboratório), mova a leitura para variável de ambiente, configure o Secret no OpenChoreo, e aí sim reabra a MR. Documente a rotação na descrição da MR para o revisor confiar.

Testes e hooks: dois gates, um antes do push, um antes do merge

inteli.

Histórico limpo e chave rotacionada resolvem o problema antigo. Falta saber se o código continua fazendo o que deveria depois de cada mudança. Dois gates, um quando você comita e outro quando você abre a MR, te protegem disso sem custo quase nenhum.

Gate 1, local (Husky pre-commit)

- Roda TruffleHog no diff staged.
- Roda `npm test` no orders-api.
- Falha rápido, feedback em segundos.
- Pode ser pulado com `--no-verify`, mas aí o gate 2 pega.

Gate 2, CI (GitLab)

- `lint:commits`, valida Conventional Commits do range da MR.
- `scan:secrets`, TruffleHog no histórico inteiro.
- `test:api`, testes do orders-api em imagem limpa.
- Gate obrigatório para merge na `main`.

Por que dois gates

O hook local é conveniência para quem está codando, o CI é a garantia formal de que nada merge sem passar. Se o hook falha mas o CI não, você tem um hook mal configurado. Se o hook passa mas o CI falha, você tem diferença de ambiente. Os dois precisam contar a mesma história.

BLOCO 3, 20 MIN

C4 e reprodutibilidade

Linguagem comum para arquitetura e o par Docker + Kubernetes como contrato de execução.

C4 Model, quatro níveis de zoom

inteli.

Proposto por Simon Brown em 2018, o C4 trata diagrama de arquitetura como mapa de zoom: a mesma cidade vista em escalas diferentes. Cada nível responde a uma pergunta diferente e fala com gente diferente.

1

Contexto

Caixa preta do sistema. Quem usa, com que outros sistemas conversa.

Stakeholders não técnicos

2

Container

Unidades que rodam em processos separados: APIs, bancos, filas, jobs.

Arquitetos e times técnicos

3

Componente

Abre um container. Módulos, responsabilidades internas, pontos de extensão.

Desenvolvedores do serviço

4

Código

Classes, funções, estruturas. Útil em pedaços críticos, raro fora deles.

Quem vai mexer naquele trecho

Na sua ponderada, o Container é o foco

É nele que aparecem os Components do OpenChoreo. Um bom C4 Container não tem caixa sem função, cada seta mostra um protocolo e cada container diz qual runtime roda dentro.

Dockerfile, contrato de reprodutibilidade

inteli.

Um Dockerfile é a receita que transforma código em imagem. Ler o Dockerfile de um projeto mostra, em 20 linhas, que runtime ele usa, quais dependências instala, como o build acontece e como a aplicação sobe.

```
# Imagem base pinada para reprodutibilidade
FROM node:20.11-alpine

WORKDIR /app

# Lockfile entra antes do código para cache
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Boas práticas essenciais

Imagens base pinadas (nunca **latest**), multi-stage para imagens finais enxutas, usuário não-root, healthcheck explícito.

O que fica de fora

Segredos, credenciais, arquivos locais do dev. Um **.dockerignore** bem feito evita vazamento acidental.

Por que isso importa para SCM

A imagem construída é um artefato versionável. O par **commit + imagem + tag** é o que permite reproduzir exatamente o que rodou em produção no dia X.

Kubernetes essencial, quatro objetos para conversar

inteli.

O Kubernetes tem dezenas de recursos. Para a Aula 02, basta entender quatro. Eles são a base de tudo o que o OpenChoreo abstrai. Quando o arquiteto falar de **Component**, vocês vão reconhecer o que está abaixo.

`kind: Pod`

Pod

Menor unidade executável. Um ou mais containers que compartilham rede e armazenamento. Efêmero por design.

`kind: Deployment`

Deployment

Mantém N réplicas de um Pod rodando. Cuida de rolling update e rollback. É o que você declara no dia a dia.

`kind: Service`

Service

Endereço estável que aponta para os Pods. Sem ele, como cada Pod tem IP próprio e volátil, ninguém se acha.

`kind: ConfigMap / Secret`

ConfigMap e Secret

Configuração e segredos externos ao Pod. Montados como variável de ambiente ou arquivo. Fator 3 do 12-factor, na prática.

A ideia por trás de tudo: estado declarado

Você escreve o estado desejado num YAML (o **manifesto**), envia ao cluster, e o Kubernetes trabalha para que a realidade bata com o manifesto. Isso é o que torna possível rodar milhares de Pods sem script e sem plantão manual. O OpenChoreo esconde a maior parte desse YAML, mas o modelo mental é esse.

SALA DE AULA INVERTIDA, 3 MIN INDIVIDUAL

Imagem ou configuração?

Vou mostrar uma aplicação e cinco itens dela. Individualmente, marquem cada item como **IMAGEM** (vai no Dockerfile), **CONFIG** (injetada pelo ambiente) ou **FORA** (fica em cofre de segredos).

1, Versão do Node que roda a app

A aplicação foi escrita em Node 20 e precisa exatamente dessa versão para funcionar.

`Muda entre ambientes? Precisa rebuild se trocar?`

2, URL do banco de dados

Em dev é localhost, em staging é db-stg.internal, em produção é db-prd.internal.

`Mesma imagem, valor diferente por ambiente.`

3, Chave da API do Stripe

Token secreto que autoriza a aplicação a cobrar cartão do cliente.

`Pode estar no git? E numa variável de ambiente no repo?`

4, Feature flag "novo checkout"

Liga ou desliga uma tela nova sem precisar subir versão. Muda no meio do dia.

`Mudança em runtime, não precisa rebuild.`

BLOCO 4, 25 MIN

OpenChoreo com a WSO2

Arquiteto da WSO2 conduz a sessão. Conceitos da plataforma, demo coletiva e ponte para a ponderada individual.

OpenChoreo, recap e o que ele absorve

inteli.

Na Aula 01 vimos que OpenChoreo é uma Internal Developer Platform (IDP) open source sobre Kubernetes, mantida pela WSO2. Hoje vamos usá-la de verdade. Antes de abrir o console, vale fixar o que a plataforma faz por você **e** o que ela exige de você.

O que a plataforma absorve

- Geração do manifesto Kubernetes, você não escreve YAML baixo nível.
- Pipeline de build e deploy entre ambientes, sem Jenkinsfile.
- Rede entre serviços, autenticação, rate limiting, observabilidade básica.
- Governança da organização, políticas da plataforma aplicam por padrão.

O que ela exige de você

- Uma aplicação que respeite o 12-factor (configuração externa).
- Um Dockerfile (ou um buildpack) que produza imagem rodável.
- Declaração do tipo do componente (Service, Web App, Task).
- Pensar nas fronteiras: o que é um componente, o que é dependência.

Como isso se liga à sua ponderada

Na defesa em vídeo, vocês vão responder: **"o que o OpenChoreo fez por mim e o que eu tive que decidir?"**. A resposta sai daqui, deste slide, não do marketing da plataforma.

Três conceitos, uma hierarquia

inteli.

OpenChoreo organiza o que é implantado em três níveis. Se vocês não fixarem essa hierarquia agora, a primeira hora no console vai ser de dúvida pura.

`kind: Project`

Project

Agrupa componentes que formam um produto ou subsistema. Fronteira de domínio. Tem quota, donos e políticas próprias.

- Exemplo: Pagamentos
- Isola custo e acesso
- Mapeia um time

`kind: Component`

Component

Unidade de código implantável. Vem com um tipo: Service, Web Application, Scheduled Task, Manual Task, API Proxy.

- Exemplo: api-cobranca
- 1 repositório, 1 componente
- Build e deploy próprios

`kind: Environment`

Environment

Onde o componente roda. Dev, staging, produção. Cada ambiente tem sua configuração, seus segredos e seu cluster-alvo.

- Exemplo: dev, staging, prod
- Promoção controlada
- Config por ambiente

Na prática: um Project contém vários Components, e cada Component é promovido por Environments distintos. A analogia com K8s: Project é Namespace, Component é Deployment, Environment é cluster.

Três recursos que aparecem na demo

inteli.

Antes da demo, mais três conceitos que vão aparecer na tela. Eles transformam Kubernetes cru em plataforma, e são os que costumam confundir quem vê OpenChoreo pela primeira vez.

Deployment Track

Caminho estável de entrega. Cada Component pode ter múltiplos tracks (ex: main, beta). Permite v1 e v2 convivendo sem que uma atropele a outra.

Managed Authentication

Autenticação provida pela plataforma. O Component declara que precisa de usuário autenticado, OpenChoreo resolve com OIDC sem escrever código.

Cell

Fronteira lógica que agrupa componentes relacionados e aplica política de rede entre eles. É como o OpenChoreo desenha limites de segurança sem precisar de service mesh manual.

No vídeo da ponderada

Se sua variante exigir login (API pública, por exemplo), o vídeo deve mostrar como Managed Auth foi configurado. Se sua variante tiver duas versões em paralelo, precisa mostrar os Deployment Tracks. A plataforma faz muito, mas você precisa saber explicar o quê ela está fazendo.

Demo coletiva, com o arquiteto da WSO2

inteli.

Agora sai do slide e entra no console. A turma inteira acompanha a implantação de uma **aplicação A** passo a passo, com o arquiteto conduzindo. O roteiro abaixo guia o que prestar atenção.

O que vai acontecer na tela

1. Criar um Project para agrupar os Components.
2. Conectar o repositório de código ao OpenChoreo.
3. Registrar o primeiro Component (tipo Service).
4. Configurar build e primeiro deploy em Environment de dev.
5. Expor o Component e validar o endpoint.

O que anotar para sua ponderada

- Em que tela apareceu cada conceito (Project, Component, Environment).
- O que foi gerado sozinho e o que o arquiteto precisou preencher.
- Como o diagrama que aparece no console se relaciona com o C4 Container.
- Quais decisões arquiteturais não foram feitas pela plataforma.

Perguntas, agora ou nunca

Essa é a hora. O arquiteto está aqui, a plataforma está aberta. Na ponderada, quem não fez pergunta hoje vai gastar horas lendo documentação depois. Use o tempo.

SALA DE AULA INVERTIDA, 4 MIN EM TRIOS

Desenhe o C4 Container do que acabamos de implantar

Com a aplicação A ainda aberta no console, em trios, rascunhem o **C4 Container** dela. Não precisa ser bonito, precisa ser fiel. Quatro perguntas guiam o rascunho.

1, Quantos containers?

Quantos Components apareceram no OpenChoreo? Cada um é um Container no C4? Tem banco ou fila junto?

`Um Component não é sempre um Container único.`

2, Como conversam?

Que protocolo liga cada par de containers? HTTP, gRPC, fila, webhook? Marque nas setas.

`Seta sem rótulo esconde decisão.`

3, Quem entra de fora?

Quem é usuário? Quem é sistema externo? O que é interno ao seu Project?

`Fronteira do sistema precisa aparecer.`

4, Onde mora a configuração?

Quais ConfigMaps e Secrets existem? Ficam no diagrama como containers ou como notas?

`C4 tem espaço para configuração como anotação.`

BLOCO 5, 10 MIN

Síntese e homework

Três checagens rápidas, lembrete da ponderada e ponte para Aula 03.

Checagem rápida, 1 de 3

inteli.

Em um time de 6 devs que entrega SaaS 2 vezes por semana, sem clientes usando versões antigas, qual estratégia de branching reduz mais o atrito?

- GitFlow, porque separa develop de main e força release formal.
- GitHub Flow ou Trunk-Based, porque main vive sempre publicável e branches são curtas.
- Forking Workflow, porque cada dev tem seu fork e a integração acontece no fim.
- Qualquer uma, a escolha é estética e não afeta entrega.

Checagem rápida, 2 de 3

inteli.

Qual afirmação sobre C4 Model e SCM está correta?

- O nível Contexto do C4 mostra classes e métodos, e vive no repositório como código.
- O nível Container do C4 mostra serviços em processos separados e deve bater com os Components que aparecem no OpenChoreo.
- Diagramas C4 dispensam versionamento, pois valem para qualquer versão do software.
- O C4 Model substitui o Dockerfile, por descrever melhor as dependências da aplicação.

Checagem rápida, 3 de 3

inteli.

No OpenChoreo, a hierarquia correta, do mais amplo para o mais estreito, é:

- Environment, Project, Component, Cell.
- Project agrupa Components, cada Component é implantado em um ou mais Environments.
- Component agrupa Projects, que são promovidos por Environments.
- Environment contém Projects, que contêm Cells, que contêm Components.

Auto-estudo para a Aula 03

inteli.

Aula 03 é sobre **métricas de software**. Antes dela, três leituras curtas e um exercício de olhar o próprio repositório com outros olhos.

Leitura, DORA metrics

Artigo da Google Cloud sobre os 4 indicadores DORA: lead time, deployment frequency, change failure rate, time to restore.

Vídeo, métricas que enganam

Palestra curta sobre vanity metrics no desenvolvimento de software (linhas de código, pontos de história).

Exercício, olhe seu repo

Antes da aula, meça uma coisa no seu repositório de projeto: lead time da sua última PR. Tempo entre abrir e mergear.

Lembrete da ponderada

Variantes serão sorteadas e enviadas por e-mail ainda hoje. Prazo no AdaLove é o fim da Sprint 01. Para quem não teve tempo de perguntar ao arquiteto, o canal da turma fica aberto.

inteli.

Até a próxima

Aula 03, Métricas de software, 28/04